

实习二：数据库约束设计

本次实习的目标是请同学们体验如何在数据库中利用各种手段完成数据库约束设计。完善的约束设计功能使得数据一致性维护以及业务规则甚至异常处理都可以统一在服务器端完成，减轻了应用处理的负担。

同学们在实现各个约束的时候，应该同时给出正负测试样例，也即符合约束以及违反约束的增删改操作。具体的约束设计任务我们分为基础、中级、高级三部分，如下：

一、基本约束设计

Emp(eno, ename, ID_number, level, position, salary, dno)

Dept(dno, dname, budget, manager)

1. 声明 eno 和 dno 是递增序列号形式的主码，长度为 4 的整型。
2. 声明 Emp 中的 dno 为参照 Dept 的外码， Dept 的 manager 为参照 Emp 的外码。
3. 测试外码定义的三种形式。
4. 限定 dname 为枚举型（数学学院、计算机学院、智能学院、电子学院、元培学院）。
5. 限定 position 为枚举型（教师、教务、会计、秘书）。
6. 限定 level 为 1 到 5，缺省为 3，salary 为 2000~200000。

二、中级约束设计

下面的题目 1 必做，题目 2 和题目 3，同学们选择其中一个实现即可。

1. 将 salary 划分为 5 个区间，每个区间对应一个 level 值，保证每个员工的工资值和他的 level 值是正确对应的，这属于行级约束。
2. 编写函数，输入员工的员工号，输出一个包含员工各方面信息的编码字符串，也即我们第二章中提到的智能码。比如 00010002199903020002，对应编码信息如下：

0001	0002	1999	03	02	0002
员工号	部门号	出生年份	级别编码	职位编码	部门领导号

3. 编写一个函数，它接收一个身份证号的前 17 位，生成一个身份证的校验码。校验码生成规则如下：

- 将身份证号码 17 位数分别乘以不同的系数。从第一位到第十七位的系数分别为：7 9 10 5 8 4 2 1 6 3 7 9 10 5 8 4 2
- 将这 17 位数字和系数相乘的结果相加
- 用加出来和除以 11，看余数是多少
- 余数 0 1 2 3 4 5 6 7 8 9 10 分别对应的最后一位身份证的号码为 1 0 X 9 8 7 6 5 4 3 2

下面是一个计算身份证校验码的算例：

身份证号码	与之相乘在数	乘积
4	7	28
3	9	27
2	10	20
8	5	40
3	8	24
1	4	4
1	2	2
9	1	9
6	6	36
4	3	12
1	7	7
1	9	9
1	10	10
5	5	25
0	8	0
8	4	32
1	2	2
乘积结果相加		287
除以11的余数		1
对应的校验码		0

三、高级约束设计

下面列出了三个题目，你可以同时选择题目 1 和题目 2，也可以单独选择题目 3。

1. 使用函数约束或者触发器来保证管理者的工资必须高于他所管理的任何一个员工
2. 使用触发器保证任何一个员工工资的变化额度，都应该体现在他所在部门的预算上面，本质上这相当于实现了一个物化视图的一致性维护机制（触发器应该考虑到员工改变工作部门的情况，从一致性维护效率的角度，完全重算当然最简单，但希望还是实现基于更新行的增量更新）
3. 使用触发器计算股票浮动盈亏。

my_stock(stock_id, volume, avg_price, profit):

表示所持有的股票编号、数量、持仓平均价格、利润

trans(trans_id, stock_id, date, price, amount, sell_or_buy):

表示一次交易的编号、股票编号、交易日期、成交价格、成交数量、买入还是卖出

使用触发器完成下面的工作：

- a) 往 trans 里面插入一条记录时，根据其是买入还是卖出，调整 my_stock 中的 volume 以及 avg_price。如果是初次插入的股票交易，就在 my_stock 中为该股票新建一条记录，profit 置为 0。注意，如果一笔卖出交易的 amount 大于 my_stock 中该股票的 volume，说明是无效的下单交易，应该加以拒绝，直接抛弃。

平均价格的计算公式：

$$avg_price = \frac{(volume * avg_price + price * amount)}{volume + amount}$$

- b) profit 的计算方式如下：每当有卖出交易发生时，将其与尽可能远的买入交易进行匹配，比如如果 trans 中现有的记录为 { (t01,s01,d01,10,1000,buy), (t02,s01,d02,12,500,buy) }, 如果现在插入 { (t03,s01,d03,11,700,sold) }, 本次交易产生的 profit=(11-10)*700, 如果再插入 { (t04,s01,d04,9,700,sold) }, 本次交易产生的 profit=(9-10)*300 + (9-12)*400 = -1500. 将每次卖出交易的 profit 都累加到 my_stock 的 profit 上。

触发器的测试数据

trans(trans_id, stock_id, date, price, amount, sell_or_buy)

(1 1 1 10 1000 B)

(2 1 2 11 500 B)

(3 1 3 12 800 S)

(4 1 4 12 1000 S)

(5 1 5 9 1000 B)

(6 1 6 12 800 S)

(7 1 7 7 800 S)